

▷ Dynamic Programming :-

It is a algorithm technique with the following properties:

- ① It is mainly an optimization over plain recursion. wherever we see a recursive solution that has repeated calls for the same inputs, we can optimize it using DP.
- ② The idea is to simply store the results of ~~the~~ subproblems so that we do not have to re-compute them when needed later.

for ex: -

```
int fib (int n) {  
    if (n <= 1)  
        return n;  
    return fib (n-1) + fib (n-2);  
}
```

↓

```
fib[0] = 0; fib[1] = 1;  
for (i = 2; i <= n; i++) {  
    fib[i] = fib[i-1] + fib[i-2];  
}  
return fib[n];
```

► why is it called Top-Down?

→ we start from fib(100)

→ Then we go downward 99, 98, 97 ...

► Tabulation (Bottom-up)

① Bottom-up approach

② stores the results of subproblems in a table.

③ Iterative implementation.

④ Entries are filled in a bottom-up manner

→ Instead of asking recursively
we already know fib(0), fib(1)
→ 0, 1, 2, 3 ...

```
for (int i = 2; i <= n; i++) {  
    dp[i] = dp[i-1] + dp[i-2];  
}  
return dp[n];
```

Memoization

① Recursive

② Top-down

③ Uses recursion stack

④ Easier to write

⑤ Computes only needed states

Tabulation

① Iterative

② Bottom-up

③ No recursion

④ usually faster

⑤ computes all states

▷ Space Optimization :

→ Notice $dp(i) = dp(i-1) + dp(i-2)$

→ we only need, previos, previos - previos.

→ So, we can store only prev 2, prev 1

```
int fib(int n) {  
    if (n <= 1)  
        return n;  
  
    int prev2 = 0;  
    int prev1 = 1;  
    for (int i = 2; i <= n; i++) {  
        int cur = prev1 + prev2;  
        prev2 = prev1;  
        prev1 = cur;  
    }  
    return prev1;  
}
```


► Memoization (Top-Down DP) :

- ① Top-down approach
- ② Stores the results of function calls in a table.
- ③ Recursive implementation
- ④ Entries are filled when needed.

⇒ we create an array: $dp[]$ initially, $-1, -1, -1, -1, -1, \dots$
⇒ meaning not solved yet

⊛ Algorithm: Need $\text{fib}(5) \rightarrow$ Is $dp[5]$ available?

⇒ code:

```
int solve (int n, vector<int> & dp) {
```

```
    if (n <= 1)
```

```
        return n;
```

```
    if (dp[n] != -1)
```

```
        return dp[n];
```

```
    dp[n] = solve(n-1, dp) + solve(n-2, dp);
```

```
    return dp[n];
```

```
}
```

```
int fib (int n) {
```

```
    vector<int> dp(n+1, -1);
```

```
    return solve(n, dp);
```

```
}
```

```
↓  
NO  
↓  
compute it  
↓  
store answer  
↓  
return answer.
```